



ÉCOLE D'INGÉNIEURS DU
LITTORAL-CÔTE-D'OPALE

Master M2 : Ingénierie des Systèmes Complexes

Détection de Fissures sur Béton :
Approche par Réseaux de Neurones Convolutifs (CNN)
et Optimisation via ResNet50

Réalisé par :
Khalil TARHDA

Encadré par :
M. Khalide JBILOU

Année Universitaire 2025 – 2026

Résumé

La maintenance des infrastructures en génie civil est un enjeu économique et sécuritaire majeur. Les méthodes traditionnelles d'inspection, basées sur l'examen visuel humain, se révèlent souvent longues, coûteuses et sujettes à la subjectivité. Ce mémoire présente une approche automatisée basée sur l'Intelligence Artificielle et plus spécifiquement le *Deep Learning* pour la détection de fissures sur des surfaces en béton.

Nous avons mis en œuvre et comparé deux architectures distinctes. Dans une première approche, nous avons conçu un **Réseau de Neurones Convolutifs (CNN)** "from scratch", composé de plusieurs couches de convolution et de pooling. Dans une seconde approche, nous avons exploité la technique de l'**Apprentissage par Transfert (Transfer Learning)** en utilisant le modèle pré-entraîné **ResNet50**.

Les modèles ont été entraînés et validés sur une base de données de 40 000 images. Les résultats démontrent la supériorité de l'approche ResNet50, qui atteint une précision de **99.85%** en seulement 10 époques, contre 99.59% pour le CNN personnalisé après 30 époques. Enfin, pour valider l'applicabilité de notre solution, nous avons développé une application web interactive permettant une analyse en temps réel.

Mots-clés : Intelligence Artificielle, Deep Learning, CNN, ResNet50, Transfer Learning, Détection de fissures, Génie Civil.

Abstract

The maintenance of civil engineering infrastructures is a major economic and safety issue. Traditional inspection methods, based on human visual examination, effectively prove to be time-consuming, expensive, and prone to subjectivity. This thesis presents an automated approach based on Artificial Intelligence and specifically *Deep Learning* for crack detection on concrete surfaces.

We implemented and compared two distinct architectures. In a first approach, we designed a custom **Convolutional Neural Network (CNN)** from scratch. In a second approach, we exploited the **Transfer Learning** technique using the pre-trained **ResNet50** model.

The models were trained and validated on a dataset of 40,000 images. The results demonstrate the superiority of the ResNet50 approach, which achieves an accuracy of **99.85%** in just 10 epochs, compared to 99.59% for the custom CNN after 30 epochs. Finally, to validate the applicability of our solution, we developed an interactive web application allowing real-time analysis.

Keywords : Artificial Intelligence, Deep Learning, CNN, ResNet50, Transfer Learning, Crack Detection, Civil Engineering.

Liste des Abréviations

- **IA** : Intelligence Artificielle
- **CNN** : Convolutional Neural Network (Réseau de Neurones Convolutifs)
- **BTP** : Bâtiment et Travaux Publics
- **SHM** : Structural Health Monitoring (Surveillance de Santé des Structures)
- **RGB** : Red, Green, Blue (Rouge, Vert, Bleu)
- **ReLU** : Rectified Linear Unit (Fonction d'activation)
- **GPU** : Graphics Processing Unit (Processeur Graphique)
- **API** : Application Programming Interface
- **MLP** : Multi-Layer Perceptron (Perceptron Multicouche)

Table des matières

Liste des Abréviations	3
Liste des Figures	6
Liste des Tableaux	7
Introduction Générale	8
1 État de l'Art et Contexte Théorique	10
1.1 Introduction à la Vision par Ordinateur	10
1.2 Les Réseaux de Neurones Convolutifs (CNN)	10
1.2.1 La Couche de Convolution	10
1.2.2 La Couche de Pooling	11
1.2.3 Fonctions d'Activation	11
1.3 Étude des Architectures CNN Avancées	11
1.3.1 VGG (Visual Geometry Group)	11
1.3.2 ResNet (Residual Networks)	11
1.3.3 EfficientNet	12
1.4 Comparaison et Choix de l'Architecture	12
1.4.1 Analyse Comparative et Sélection de l'Architecture	13
1.5 Le Transfert d'Apprentissage (Transfer Learning)	14
1.5.1 Concept Fondamental	14
1.5.2 La Puissance d'ImageNet	14
1.5.3 Pourquoi cela fonctionne-t-il pour le béton ?	14
1.5.4 Stratégie de Mise en Œuvre : Feature Extraction vs Fine-Tuning . .	15
2 Méthodologie et Outils	16
2.1 Environnement Technique	16
2.1.1 Langage et Bibliothèques	16
2.1.2 Ressources Matérielles (Hardware)	16
2.1.3 Déploiement	17
2.2 Le Jeu de Données (Dataset)	17
2.2.1 Composition et Équilibre	17
2.3 Pipeline de Prétraitement des Données	17
2.3.1 Redimensionnement (Resizing)	17
2.3.2 Normalisation (Rescaling)	18
2.3.3 Stratégie de Séparation (Train / Validation)	18
2.3.4 Optimisation I/O (Input/Output)	18
2.4 Métriques d'Évaluation	19

2.4.1	La Précision (Accuracy)	19
2.4.2	La Fonction de Perte (Binary Crossentropy)	19
3	Implémentation et Analyse des Résultats	20
3.1	Protocole Expérimental	20
3.2	Approche 1 : Création d'un CNN Personnalisé	20
3.2.1	Architecture détaillée du CNN Personnalisé	20
3.2.2	Résultats Expérimentaux (CNN)	22
3.3	Approche 2 : Optimisation via ResNet50	22
3.3.1	Architecture et Stratégie de Transfert	22
3.3.2	Résultats Expérimentaux (ResNet50)	23
3.4	Étude Comparative	24
3.4.1	Bilan de la comparaison	25
4	Validation et Déploiement Industriel	26
4.1	Validation Finale sur le Jeu de Test	26
4.1.1	Analyse de la Matrice de Confusion	26
4.1.2	Métriques de Performance Détaillées	27
4.2	Architecture de l'Application de Déploiement	28
4.2.1	Fonctionnement Technique	28
4.3	Démonstration et Scénarios d'Usage	29
4.3.1	Scénario 1 : Validation de structure saine	29
4.3.2	Scénario 2 : Alerte Fissure	30
4.4	Discussion et Limites	31
	Conclusion Générale	32
	Bibliographie et Webographie	34

Table des figures

1.1	Schéma théorique d'un bloc résiduel (Skip Connection)	12
3.1	Courbe de Précision (CNN)	22
3.2	Courbe de Perte (CNN)	22
3.3	Précision (ResNet50)	23
3.4	Perte (ResNet50)	23
3.5	Comparaison des Précisions (CNN vs ResNet)	24
3.6	Comparaison des Erreurs (CNN vs ResNet)	25
4.1	Matrice de Confusion (Modèle ResNet50)	27
4.2	Interface Web : Diagnostic d'un béton Sain (Indicateur Vert)	29
4.3	Interface Web : Détection d'une Fissure (Alerte Rouge)	30

Liste des tableaux

1.1	Comparaison des architectures CNN (Poids sur ImageNet)	12
2.1	Répartition des classes du Dataset (Source : Kaggle)	17
3.1	Résumé de l'architecture du modèle CNN "From scratch"	21
4.1	Rapport de Classification (Test Set)	27

Introduction Générale

Contexte du Projet

Le secteur du Bâtiment et des Travaux Publics (BTP) connaît aujourd'hui une transformation numérique sans précédent, souvent qualifiée de "Construction 4.0". Au cœur de cette révolution, la maintenance des infrastructures existantes (ponts, barrages, immeubles d'habitation, centrales nucléaires) représente un défi colossal à l'échelle mondiale. Le béton, matériau de construction le plus utilisé sur Terre, est un matériau vivant sujet à diverses pathologies au cours de son cycle de vie.

Parmi ces pathologies, la fissuration est la plus courante et la plus révélatrice. Une fissure dans le béton n'est jamais simplement un défaut esthétique ; elle est souvent le symptôme précurseur de problèmes structurels plus graves tels que la corrosion des armatures internes, la carbonatation ou des défaillances mécaniques dues à la fatigue des matériaux.

Dans une logique de maintenance prédictive, la surveillance de la santé structurelle (SHM - Structural Health Monitoring) est devenue une priorité pour les gestionnaires d'ouvrages. Il s'agit d'intervenir "au bon moment" : ni trop tôt (coûts inutiles), ni trop tard (risque d'effondrement ou de réparations lourdes).

Problématique

Malgré les enjeux sécuritaires et financiers, la grande majorité des inspections de structures en béton sont encore réalisées de manière traditionnelle. Des ingénieurs ou techniciens qualifiés se déplacent sur site pour examiner visuellement les surfaces, mesurer l'ouverture des fissures et noter leur évolution.

Cette approche manuelle présente aujourd'hui des limites critiques qui freinent l'efficacité de la maintenance :

1. **Subjectivité et Variabilité** : Le diagnostic dépend fortement de l'expertise de l'inspecteur. La fatigue, les conditions d'éclairage ou le biais cognitif peuvent mener à des interprétations divergentes pour une même pathologie.
2. **Coûts et Logistique** : L'inspection de grands ouvrages d'art (comme les viaducs ou les tours de refroidissement) nécessite des moyens d'accès lourds (nacelles, échafaudages, cordistes) et mobilise du personnel sur de longues périodes, engendrant des coûts d'exploitation élevés.
3. **Sécurité et Accessibilité** : Certaines zones des ouvrages sont difficiles d'accès, voire dangereuses pour les opérateurs humains (hauteur, zones irradiées, tunnels).
4. **Volume de Données** : Avec le vieillissement global des infrastructures, le nombre de surfaces à inspecter dépasse désormais les capacités humaines disponibles.

Face à ces constats, la problématique centrale de ce mémoire est la suivante : ***Comment l'Intelligence Artificielle et le Deep Learning peuvent-ils automatiser, fiabiliser et sécuriser le processus de détection des fissures sur les infrastructures en béton ?***

Approche Proposée

Pour répondre à cette question, nous proposons d'exploiter la puissance de la Vision par Ordinateur (Computer Vision). L'idée est de remplacer l'œil humain par des algorithmes capables d'apprendre à reconnaître les caractéristiques visuelles complexes d'une fissure (forme, texture, contraste) à partir d'images numériques.

Nous nous concentrerons sur deux approches technologiques distinctes pour évaluer la meilleure stratégie industrielle : la création d'un modèle sur-mesure (Custom CNN) et l'adaptation d'un modèle pré-existant très performant (ResNet50).

Objectifs du Projet

L'objectif principal de ce travail est de concevoir une chaîne de traitement complète, allant de la donnée brute à l'outil d'aide à la décision. Les objectifs spécifiques sont les suivants :

- **Constitution des données** : Préparer et prétraiter un jeu de données représentatif (40 000 images) pour garantir un apprentissage non biaisé.
- **Modélisation "From Scratch"** : Développer une architecture de Réseau de Neurones Convolutifs (CNN) personnalisée pour établir une performance de référence (Baseline).
- **Optimisation par Transfer Learning** : Implémenter une stratégie d'apprentissage par transfert en utilisant l'architecture profonde ResNet50 pour améliorer la précision et réduire le temps de calcul.
- **Analyse Comparative** : Mener une étude rigoureuse des performances des deux modèles (Précision, Rappel, Matrices de confusion, Courbes de convergence).
- **Déploiement Applicatif** : Intégrer le modèle le plus performant dans une interface utilisateur web (via Streamlit) pour simuler un cas d'usage réel sur le terrain.

Plan du Mémoire

Ce document s'articule autour de quatre chapitres principaux :

- Le **Chapitre 1** dresse l'état de l'art. Il présente les concepts fondamentaux des réseaux de neurones, du Deep Learning et justifie le choix des architectures CNN et ResNet.
- Le **Chapitre 2** détaille la méthodologie adoptée. Il décrit l'environnement technique, les caractéristiques du jeu de données et les étapes cruciales de prétraitement.
- Le **Chapitre 3** est consacré à l'implémentation et aux résultats. Nous y analysons les performances comparées de nos deux approches expérimentales.
- Enfin, le **Chapitre 4** présente la phase de validation finale et le déploiement de l'application de diagnostic, avant de conclure sur les perspectives d'avenir.

Chapitre 1

État de l'Art et Contexte Théorique

1.1 Introduction à la Vision par Ordinateur

La vision par ordinateur (Computer Vision) est un domaine de l'intelligence artificielle qui vise à permettre aux machines d'analyser, de comprendre et d'interpréter des images numériques.

Historiquement, la reconnaissance de motifs reposait sur des méthodes artisanales ("Hand-crafted features"). Des algorithmes comme SIFT (Scale-Invariant Feature Transform) ou les détecteurs de contours de Canny nécessitaient une expertise humaine pour définir les caractéristiques à extraire (bords, coins, textures). Ces méthodes montraient leurs limites face à la variabilité des images réelles (changements d'éclairage, rotations, occlusions).

L'avènement du **Deep Learning** (apprentissage profond) vers 2012, avec la victoire d'AlexNet au concours ImageNet, a marqué une rupture technologique. Désormais, ce n'est plus l'ingénieur qui définit les caractéristiques, mais le réseau de neurones qui les apprend automatiquement à partir des données brutes.

1.2 Les Réseaux de Neurones Convolutifs (CNN)

Les Réseaux de Neurones Convolutifs (CNN ou ConvNets) constituent l'architecture fondamentale pour le traitement d'images. Ils s'inspirent biologiquement de l'organisation du cortex visuel animal, où des neurones individuels réagissent à des stimuli dans des champs réceptifs restreints.

Un CNN est généralement composé d'une alternance de trois types de couches : la convolution, le pooling et les couches entièrement connectées.

1.2.1 La Couche de Convolution

C'est le cœur du réseau. Contrairement à un réseau de neurones classique (MLP) où chaque neurone est connecté à tous les neurones de la couche précédente, la couche de convolution utilise des filtres (ou noyaux/kernels) de petite taille (ex : 3×3 ou 5×5).

Mathématiquement, pour une image d'entrée I et un filtre K de taille $m \times n$, la convolution discrète ($I * K$) au point (i, j) s'exprime par :

$$(I * K)(i, j) = \sum_{u=0}^{m-1} \sum_{v=0}^{n-1} I(i+u, j+v) \cdot K(u, v) \quad (1.1)$$

Cette opération permet de générer des *cartes de caractéristiques* (Feature Maps). Les premières couches détectent des éléments simples (lignes, courbes), tandis que les couches profondes combinent ces éléments pour reconnaître des objets complexes (fissures, visages).

1.2.2 La Couche de Pooling

Le pooling sert à réduire progressivement la taille spatiale de la représentation (Down-sampling) pour diminuer le nombre de paramètres et le coût de calcul, tout en contrôlant le surapprentissage (Overfitting). La méthode la plus courante est le **Max Pooling**, qui ne conserve que la valeur maximale dans une fenêtre donnée.

1.2.3 Fonctions d'Activation

Pour introduire de la non-linéarité dans le modèle (essentielle pour apprendre des frontières de décision complexes), on applique une fonction d'activation après chaque convolution. La plus utilisée est la fonction **ReLU** (Rectified Linear Unit) :

$$f(x) = \max(0, x) \quad (1.2)$$

Elle permet d'accélérer la convergence par rapport aux anciennes fonctions comme la Sigmoid ou Tanh.

1.3 Étude des Architectures CNN Avancées

Pour concevoir un réseau adapté à la reconnaissance précise de fissures, il est nécessaire d'étudier les architectures qui ont marqué l'état de l'art. Nous nous sommes penchés sur trois familles majeures.

1.3.1 VGG (Visual Geometry Group)

Proposé par l'Université d'Oxford en 2014, le réseau VGG (notamment VGG-16 et VGG-19) se caractérise par sa simplicité et sa profondeur.

- **Concept** : Utilisation exclusive de très petits filtres de convolution (3×3). L'idée est qu'une succession de deux filtres 3×3 a le même champ réceptif qu'un filtre 5×5 , mais avec moins de paramètres et plus de non-linéarité.
- **Limites** : VGG est extrêmement lourd en mémoire. VGG-16 possède environ 138 millions de paramètres, ce qui rend son entraînement lent et le déploiement sur des appareils mobiles difficile.

1.3.2 ResNet (Residual Networks)

Introduit par Microsoft en 2015, ResNet a résolu le problème de la "disparition du gradient" (Vanishing Gradient) qui empêchait d'entraîner des réseaux très profonds.

- **Concept** : L'innovation clé est le **bloc résiduel** (Residual Block) avec une connexion "skip" (saut de connexion). Au lieu d'apprendre une fonction $H(x)$, le réseau apprend le résidu $F(x) = H(x) - x$. La sortie du bloc devient $F(x) + x$.
- **Avantage** : Cela permet au gradient de circuler plus facilement lors de la rétro-propagation. On peut ainsi construire des réseaux de 50, 101, voire 152 couches.

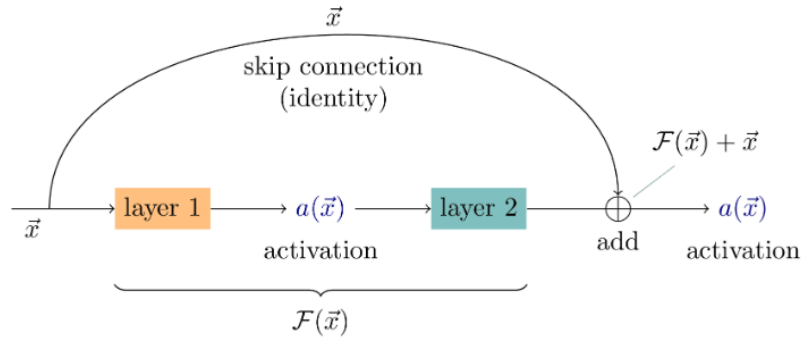


FIGURE 1.1 – Schéma théorique d'un bloc résiduel (Skip Connection)

1.3.3 EfficientNet

Plus récent (2019), EfficientNet propose une méthode d'optimisation appelée "Compound Scaling". Au lieu d'augmenter arbitrairement la profondeur (comme ResNet) ou la largeur du réseau, EfficientNet augmente uniformément la profondeur, la largeur et la résolution de l'image via un coefficient composé. Cela offre un excellent compromis performance/coût de calcul, mais l'architecture est plus complexe à implémenter "from scratch".

1.4 Comparaison et Choix de l'Architecture

Le tableau ci-dessous résume les caractéristiques des modèles étudiés pour notre tâche de détection de fissures.

TABLE 1.1 – Comparaison des architectures CNN (Poids sur ImageNet)

Architecture	Année	Profondeur	Paramètres (Millions)	Taille (Mo)	Précision (Top-5)
VGG-16	2014	16 couches	138.3 M	528	90.1 %
ResNet50	2015	50 couches	25.6 M	98	92.1 %
InceptionV3	2015	48 couches	23.8 M	92	93.7 %
EfficientNet-B0	2019	Varie	5.3 M	29	93.3 %

1.4.1 Analyse Comparative et Sélection de l'Architecture

Conformément aux objectifs du projet, nous avons étudié trois familles d'architectures CNN emblématiques pour déterminer laquelle offrirait le meilleur compromis entre précision, rapidité d'apprentissage et facilité de déploiement pour la détection de fissures.

1. Analyse de VGG-16 (L'approche historique)

Le réseau VGG-16 est réputé pour sa simplicité structurelle (empilement de convolutions 3×3).

- **Avantages** : Architecture très linéaire et facile à comprendre. Excellente extraction de caractéristiques de bas niveau.
- **Inconvénients majeurs** : C'est un modèle excessivement lourd. Avec environ **138 millions de paramètres**, le fichier de poids dépasse les 500 Mo.
- **Décision** : Nous avons écarté VGG-16 car son coût computationnel est prohibitif pour une application industrielle (inspection par drone ou mobile) et il présente un risque élevé de sur-apprentissage sur un dataset de 40 000 images sans une régularisation massive.

2. Analyse d'EfficientNet (L'approche moderne)

La famille EfficientNet (B0 à B7) utilise une méthode de "Compound Scaling" pour optimiser simultanément la largeur, la profondeur et la résolution du réseau.

- **Avantages** : Il offre théoriquement le meilleur ratio précision/paramètres de l'état de l'art actuel.
- **Inconvénients** : Son architecture est complexe et fortement dépendante d'hyperparamètres spécifiques (résolution d'entrée stricte). De plus, son implémentation en Transfer Learning peut parfois se révéler instable lors du "Fine-Tuning" si les taux d'apprentissage ne sont pas parfaitement calibrés.
- **Décision** : Bien que prometteur, EfficientNet a été écarté pour privilégier une architecture plus robuste et standardisée pour ce prototype.

3. Sélection de ResNet50 (Le compromis optimal)

Finalement, notre choix s'est porté sur ResNet50. Ce modèle représente le point d'équilibre idéal pour notre problématique de Génie Civil :

1. **Résolution du problème de profondeur** : Grâce à ses blocs résiduels et ses connexions (voir Figure 1.1), ResNet permet d'entraîner un réseau profond (50 couches) sans subir la disparition du gradient. C'est crucial pour capter la texture fine des fissures qui peut se perdre dans des réseaux classiques.
2. **Efficacité Paramétrique** : Avec seulement **25 millions de paramètres** (soit 5 fois moins que VGG), il est beaucoup plus rapide à entraîner et plus léger à déployer, tout en offrant une précision supérieure sur ImageNet.
3. **Standard Industriel** : ResNet50 est documenté de manière exhaustive et constitue le standard "de facto" pour la classification d'images industrielles, garantissant une reproductibilité et une stabilité maximales pour notre projet.

1.5 Le Transfert d'Apprentissage (Transfer Learning)

1.5.1 Concept Fondamental

L'entraînement d'un Réseau de Neurones Convolutif profond (Deep CNN) à partir de zéro ("from scratch") présente deux obstacles majeurs :

1. **Le volume de données** : Il faut des millions d'images annotées pour éviter le sur-apprentissage.
2. **Le coût computationnel** : La convergence des poids nécessite des semaines de calcul sur des grappes de GPU puissants.

Le **Transfert d'Apprentissage** permet de contourner ces limitations. Le principe repose sur une intuition humaine simple : nous n'apprenons pas chaque nouvelle tâche depuis le néant. Un musicien qui sait jouer du piano apprendra l'orgue beaucoup plus vite qu'un novice, car il transfère ses connaissances musicales (solfège, rythme, dextérité).

En Deep Learning, cela consiste à réutiliser un modèle pré-entraîné sur une tâche source (très vaste) pour résoudre une tâche cible (notre problème spécifique).

1.5.2 La Puissance d'ImageNet

Dans notre cas, nous utilisons des modèles (comme ResNet50) pré-entraînés sur la base de données **ImageNet**. Celle-ci contient plus de 1.4 million d'images réparties en 1000 classes (animaux, véhicules, objets du quotidien...). Bien que notre objectif soit de détecter des fissures sur du béton (ce qui ne fait pas partie des 1000 classes ImageNet), ce pré-entraînement est inestimable.

1.5.3 Pourquoi cela fonctionne-t-il pour le béton ?

Les CNN apprennent de manière hiérarchique :

- **Les premières couches (Bas niveau)** : Elles détectent des caractéristiques universelles comme les lignes droites, les courbes, les coins et les gradients de couleur. Ces éléments sont présents aussi bien dans une photo de chat que dans une photo de fissure.
- **Les couches intermédiaires** : Elles détectent des textures et des motifs géométriques simples.
- **Les dernières couches (Haut niveau)** : Elles assemblent ces motifs pour reconnaître des objets complexes spécifiques (oreilles de chat, roues de voiture).

L'idée du transfert est de conserver les premières couches (qui savent déjà "voir") et de ne modifier que les dernières couches pour qu'elles apprennent à interpréter ces caractéristiques dans le contexte du béton.

1.5.4 Stratégie de Mise en Œuvre : Feature Extraction vs Fine-Tuning

Nous avons adopté une approche hybride pour adapter ResNet50 :

1. **Extraction de Caractéristiques (Feature Extraction)** : Nous chargeons le modèle en excluant la "tête" de classification (les dernières couches denses). Le "corps" du réseau (Backbone) est figé (*Frozen*), c'est-à-dire que ses poids ne sont pas modifiés lors de la rétropropagation. Il agit comme un extracteur de caractéristiques fixe.
2. **Adaptation (Custom Head)** : Nous greffons de nouvelles couches denses (Fully Connected) initialisées aléatoirement à la sortie du backbone. Ce sont les seules couches que nous entraînons. Elles apprennent à associer les textures extraites par ResNet50 aux classes "Sain" ou "Fissuré".

Cette méthode permet d'obtenir des résultats excellents avec un temps d'entraînement réduit drastiquement, car le réseau n'a pas besoin de réapprendre à détecter un bord ou une texture rugueuse : il le sait déjà.

Chapitre 2

Méthodologie et Outils

Ce chapitre présente l'environnement technique, les ressources matérielles et logicielles utilisées, ainsi que la stratégie de préparation des données adoptée pour mener à bien ce projet de détection de fissures.

2.1 Environnement Technique

Le choix des outils est déterminant pour la réussite d'un projet de Deep Learning. Nous avons opté pour une stack technologique moderne, open-source et largement utilisée dans l'industrie.

2.1.1 Langage et Bibliothèques

- **Python 3.10** : Choisi pour sa richesse en bibliothèques de Data Science et sa syntaxe claire.
- **TensorFlow 2.15 & Keras** : Nous avons utilisé TensorFlow comme backend de calcul et Keras comme API de haut niveau. Keras permet un prototypage rapide des architectures de réseaux de neurones (CNN et ResNet) grâce à sa modularité.
- **NumPy** : Utilisé pour la manipulation efficace des tableaux multidimensionnels (tenseurs) représentant les images.
- **Matplotlib & Seaborn** : Ces bibliothèques nous ont permis de visualiser les résultats, notamment les courbes d'apprentissage (Loss/Accuracy) et les matrices de confusion.

2.1.2 Ressources Matérielles (Hardware)

L'entraînement de réseaux de neurones profonds nécessite une grande puissance de calcul parallèle.

- **Google Colab** : Nous avons utilisé l'environnement cloud de Google pour bénéficier gratuitement d'un accès aux GPU (Graphics Processing Units).
- **GPU T4 (Tesla)** : L'accélération GPU est indispensable. Contrairement au CPU (processeur classique) qui traite les tâches séquentiellement, le GPU possède des milliers de cœurs permettant de réaliser les opérations matricielles (convolutions) en parallèle, réduisant le temps d'entraînement de plusieurs heures à quelques minutes.

2.1.3 Déploiement

Streamlit a été retenu pour la phase de déploiement. C'est un framework Python qui permet de transformer un script de Data Science en une application web interactive sans nécessiter de compétences avancées en développement web (HTML/CSS).

2.2 Le Jeu de Données (Dataset)

La qualité du modèle dépend directement de la qualité des données ("Garbage In, Garbage Out"). Pour ce projet, nous avons utilisé un jeu de données public de référence, disponible sur la plateforme de science des données **Kaggle**.

Il s'agit du dataset intitulé "*Concrete Crack Images for Classification*"¹, largement utilisé dans la littérature scientifique pour l'évaluation des algorithmes de vision par ordinateur appliqués au génie civil.

2.2.1 Composition et Équilibre

Le dataset est composé de **40 000 images** couleurs (RGB) de surfaces en béton, réparties en deux classes distinctes :

TABLE 2.1 – Répartition des classes du Dataset (Source : Kaggle)

Classe	Description	Nombre d'images
Positive	Images contenant une ou plusieurs fissures	20 000
Negative	Images de béton sain (sans défaut)	20 000
Total		40 000

Importance de l'équilibre : Cet équilibre parfait (50% / 50%) est crucial. Si le dataset contenait 90% de béton sain, le modèle pourrait obtenir 90% de précision simplement en prédisant "Sain" tout le temps, sans jamais apprendre à détecter une fissure (phénomène de biais de classe). L'équilibre garantit que le modèle apprend réellement les caractéristiques visuelles des fissures.

2.3 Pipeline de Prétraitement des Données

Avant d'être injectées dans le réseau de neurones, les images brutes subissent une série de transformations pour optimiser l'apprentissage.

2.3.1 Redimensionnement (Resizing)

Les images d'origine peuvent avoir des tailles variées. Les réseaux de neurones à couches entièrement connectées (Dense Layers) nécessitent un vecteur d'entrée de taille fixe. Nous avons standardisé toutes les images à une résolution de :

$$227 \times 227 \text{ pixels} \times 3 \text{ canaux (Rouge, Vert, Bleu)}$$

1. Source : <https://www.kaggle.com/datasets/arunrk7/surface-crack-detection>

L'utilisation des 3 canaux RVB est essentielle car la teinte des fissures (souvent brunâtre ou sombre) contraste avec le gris du béton, une information qui serait perdue en niveaux de gris simples.

2.3.2 Normalisation (Rescaling)

Les valeurs de pixels d'une image numérique varient traditionnellement entre 0 (noir) et 255 (blanc). Cependant, les réseaux de neurones convergent plus rapidement et plus stablement lorsque les données d'entrée sont petites (proches de 0 et 1). Nous avons appliqué une couche de *Rescaling* :

$$x_{norm} = \frac{x}{255} \quad (2.1)$$

Où x est la valeur du pixel original et x_{norm} la valeur normalisée entre $[0, 1]$.

2.3.3 Stratégie de Séparation (Train / Validation)

Nous avons divisé notre jeu de données en deux sous-ensembles distincts :

- **Entraînement (Training Set - 80%)** : 32 000 images utilisées par le modèle pour ajuster ses poids par rétropropagation du gradient.
- **Validation (Validation Set - 20%)** : 8 000 images utilisées à la fin de chaque époque pour évaluer la performance du modèle sur des données inconnues.

Cette séparation est indispensable pour détecter le phénomène de sur-apprentissage (Overfitting), qui se produit lorsque le modèle apprend "par cœur" les images d'entraînement mais échoue sur de nouvelles images.

2.3.4 Optimisation I/O (Input/Output)

Pour éviter que le GPU (très rapide) n'attende le chargement des images depuis le disque dur (plus lent), nous avons mis en place un pipeline de données optimisé avec TensorFlow :

- **Batching** : Les images sont regroupées par lots de 32 (`batch_size=32`). Cela permet de lisser la descente de gradient.
- **Prefetching** : Pendant que le GPU traite le lot N , le CPU prépare et charge en mémoire le lot $N + 1$. Nous avons utilisé la commande `AUTOTUNE` pour que TensorFlow gère dynamiquement cette mémoire tampon.

2.4 Métriques d'Évaluation

Pour juger la qualité de nos modèles, nous surveillons deux indicateurs principaux durant l'apprentissage.

2.4.1 La Précision (Accuracy)

C'est le pourcentage de prédictions correctes par rapport au nombre total d'images.

$$Accuracy = \frac{VP + VN}{VP + VN + FP + FN} \quad (2.2)$$

(Où VP=Vrais Positifs, VN=Vrais Négatifs, FP=Faux Positifs, FN=Faux Négatifs.)

2.4.2 La Fonction de Perte (Binary Crossentropy)

C'est la métrique mathématique que le réseau cherche à minimiser (la "Loss"). Pour une classification binaire, elle pénalise fortement les prédictions confiantes mais fausses.

$$Loss = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (2.3)$$

Où y_i est la vraie étiquette (0 ou 1) et $\hat{y}_i$ est la probabilité prédite par le modèle. Plus cette valeur est proche de 0, meilleur est le modèle.

Chapitre 3

Implémentation et Analyse des Résultats

Dans ce chapitre, nous détaillons la mise en œuvre technique de nos deux approches expérimentales. L'objectif est de comparer une architecture conçue manuellement (Custom CNN) face à une architecture issue de l'état de l'art (ResNet50).

3.1 Protocole Expérimental

Le jeu de données est chargé depuis le répertoire dataset. Pour garantir la reproductibilité des résultats, nous avons défini les paramètres suivants :

- **Dimensions d'entrée** : 227×227 pixels (3 canaux RVB).
- **Taille du Batch** : 32 images par lot (compromis idéal pour la mémoire GPU).
- **Séparation (Split)** : 80% pour l'entraînement et 20% pour la validation.

3.2 Approche 1 : Création d'un CNN Personnalisé

3.2.1 Architecture détaillée du CNN Personnalisé

Pour établir une performance de référence (Baseline), nous avons conçu une architecture séquentielle de type "Pyramidal". Le principe est d'augmenter progressivement la profondeur des cartes de caractéristiques (nombre de filtres) tout en réduisant leur dimension spatiale.

Le réseau est constitué de deux parties distinctes :

1. L'Extracteur de Caractéristiques (Feature Extractor)

Cette partie agit comme l'œil du réseau. Elle est composée de trois blocs de convolution successifs :

- **Bloc 1 (Entrée)** : Il reçoit l'image brute ($227 \times 227 \times 3$). Il applique 32 filtres pour détecter les caractéristiques de bas niveau (contours, lignes).
- **Bloc 2 (Intermédiaire)** : Il applique 64 filtres pour combiner les traits simples en formes géométriques.
- **Bloc 3 (Profond)** : Il applique 128 filtres pour capter des textures complexes spécifiques aux fissures.

Chaque bloc est suivi d'une couche de *MaxPooling* (2×2) qui divise par deux la taille de l'image, réduisant ainsi le nombre de calculs et le risque de sur-apprentissage.

2. Le Classifieur (Classifier)

Cette partie agit comme le cerveau. Elle prend les caractéristiques extraites, les met à plat (vecteur 1D) et utilise un réseau de neurones dense pour prendre la décision finale.

TABLE 3.1 – Résumé de l'architecture du modèle CNN "From scratch"

Type de Couche	Configuration	Activation	Rôle
Entrée (Input)	$227 \times 227 \times 3$ (RGB)	-	Normalisation (1./255)
Conv2D (Bloc 1)	32 filtres, noyau 3×3	ReLU	Détection contours
MaxPooling2D	Fenêtre 2×2	-	Réduction dimension
Conv2D (Bloc 2)	64 filtres, noyau 3×3	ReLU	Détection formes
MaxPooling2D	Fenêtre 2×2	-	Réduction dimension
Conv2D (Bloc 3)	128 filtres, noyau 3×3	ReLU	Détection textures
MaxPooling2D	Fenêtre 2×2	-	Réduction dimension
Flatten	-	-	Mise à plat
Dense (Cachée)	128 neurones	ReLU	Interprétation
Dense (Sortie)	1 neurone	Sigmoid	Probabilité (0-1)

Choix des Hyperparamètres :

- **Fonction d'activation ReLU** : Choisie pour toutes les couches cachées car elle accélère la convergence et évite le problème de disparition du gradient.
- **Optimiseur Adam** : Utilisé pour sa capacité à adapter le taux d'apprentissage automatiquement.
- **Fonction de perte** : *Binary Crossentropy*, car il s'agit d'une classification binaire stricte (Sain vs Fissuré).

3.2.2 Résultats Expérimentaux (CNN)

Le modèle a été entraîné sur **30 époques**. Les courbes ci-dessous illustrent l'évolution de la précision et de l'erreur durant l'apprentissage.

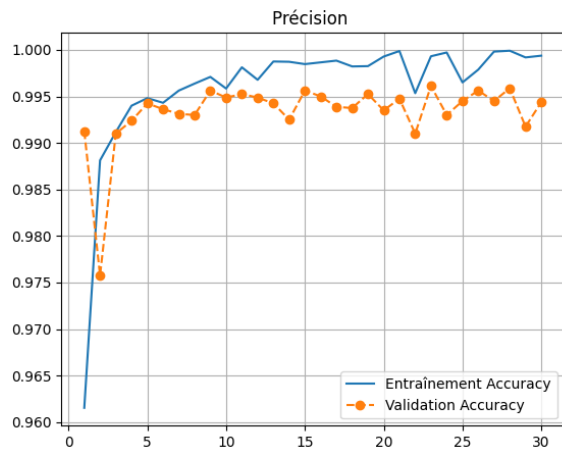


FIGURE 3.1 – Courbe de Précision (CNN)

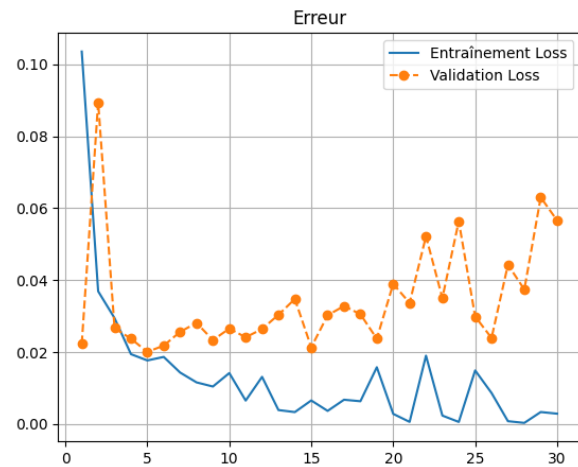


FIGURE 3.2 – Courbe de Perte (CNN)

Analyse : Nous atteignons une précision finale de **99.44%**. C'est un excellent résultat pour un modèle simple. Cependant, on remarque que l'apprentissage est relativement long et nécessite 30 époques pour se stabiliser complètement.

3.3 Approche 2 : Optimisation via ResNet50

Pour accélérer l'apprentissage et tenter d'atteindre une précision encore plus élevée, nous avons utilisé la technique du **Transfer Learning**.

3.3.1 Architecture et Stratégie de Transfert

L'implémentation de ResNet50 s'est faite en quatre étapes clés, visant à adapter ce "cerveau" pré-entraîné à notre problématique spécifique de fissures.

1. **Chargement du modèle de base (Backbone) :** Nous avons importé ResNet50 avec les poids *ImageNet* (connaissances acquises sur 1000 classes d'objets). Le paramètre `include_top=False` a été utilisé pour supprimer la dernière couche de classification d'origine.
2. **Congélation des poids (Freezing) :** Nous avons rendu le modèle de base non-entraînable (`base_model.trainable = False`). Cela permet de conserver les filtres de détection de textures et de formes appris sur ImageNet sans les détériorer lors du nouvel entraînement.
3. **Construction de la nouvelle tête de classification :** Nous avons ajouté nos propres couches au-dessus de ResNet50 :
 - **GlobalAveragePooling2D :** Pour résumer les cartes de caractéristiques en un vecteur simple.
 - **Dense (128, ReLU) :** Une couche intermédiaire pour l'interprétation.
 - **Dropout (0.2) :** Une technique de régularisation qui désactive aléatoirement 20% des neurones pour empêcher le sur-apprentissage.

— **Dense (1, Sigmoid)** : La couche finale pour le verdict binaire.

4. **Compilation et Hyperparamètres** : Le modèle a été compilé avec l'optimiseur **Adam**. Un taux d'apprentissage (Learning Rate) très faible de **0.0001** (10^{-4}) a été choisi pour ajuster les poids en douceur sans "brusquer" le modèle.

3.3.2 Résultats Expérimentaux (ResNet50)

Contrairement au CNN précédent, ce modèle n'a nécessité que **10 époques** pour converger vers une solution optimale.

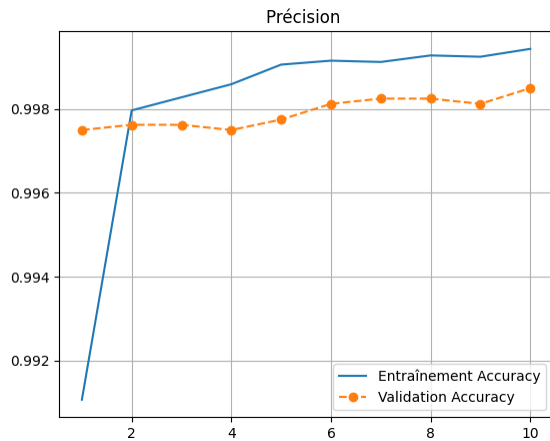


FIGURE 3.3 – Précision (ResNet50)

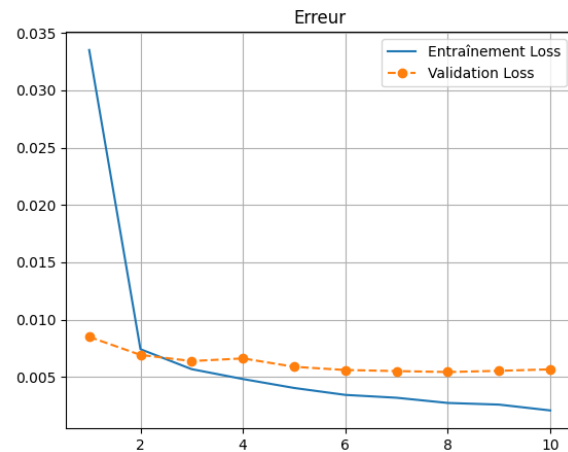


FIGURE 3.4 – Perte (ResNet50)

Performance : La précision monte à **99.85%** en seulement 10 époques. La courbe de perte (Loss) descend beaucoup plus vite et de manière plus lisse que celle du CNN personnalisé.

3.4 Étude Comparative

La comparaison directe des deux approches met en évidence la supériorité de l'apprentissage par transfert dans ce contexte.

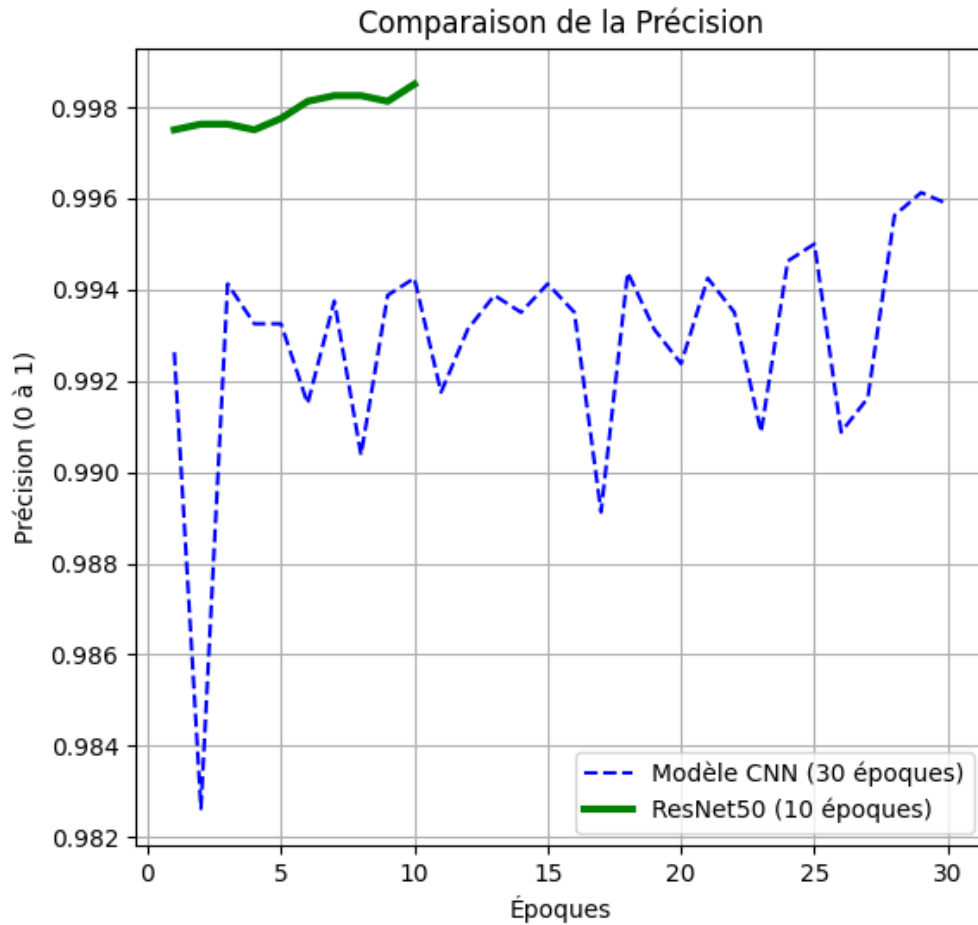


FIGURE 3.5 – Comparaison des Précisions (CNN vs ResNet)

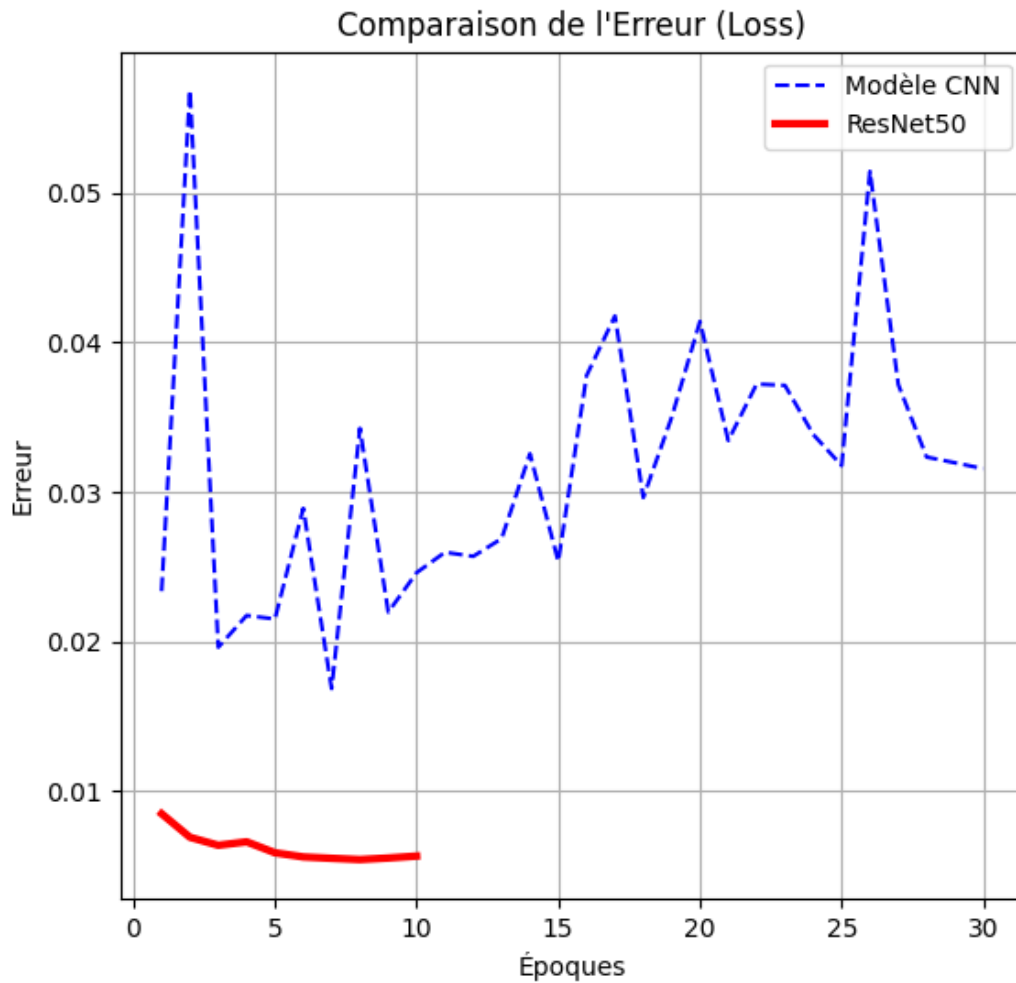


FIGURE 3.6 – Comparaison des Erreurs (CNN vs ResNet)

3.4.1 Bilan de la comparaison

1. **Vitesse de Convergence** : Le ResNet50 (courbe orange) atteint un niveau de performance élevé dès les premières époques, grâce à ses poids pré-entraînés sur ImageNet.
2. **Stabilité** : La courbe d'erreur du ResNet50 est plus stable et plus basse que celle du CNN (courbe bleue), ce qui indique une meilleure généralisation et moins de risques de surapprentissage, notamment grâce à la couche de *Dropout*.
3. **Précision Finale** : Avec 99.85% contre 99.44%, ResNet50 offre une fiabilité accrue, critique pour la sécurité des ouvrages d'art.

Chapitre 4

Validation et Déploiement Industriel

Après avoir entraîné et optimisé nos modèles, l'étape finale consiste à valider leur robustesse sur des données jamais vues et à proposer une solution concrète pour leur utilisation sur le terrain. Ce chapitre détaille les performances finales du modèle ResNet50 et présente l'architecture de l'application de diagnostic développée.

4.1 Validation Finale sur le Jeu de Test

Pour cette phase de validation, nous avons isolé un jeu de test de **8000 images** (4080 saines et 3920 fissurées) qui n'ont jamais été présentées au modèle durant l'entraînement. L'objectif est de simuler le comportement de l'IA face à de nouveaux murs de béton réels.

4.1.1 Analyse de la Matrice de Confusion

La matrice de confusion est un outil fondamental qui permet de confronter les prédictions du modèle à la réalité terrain.

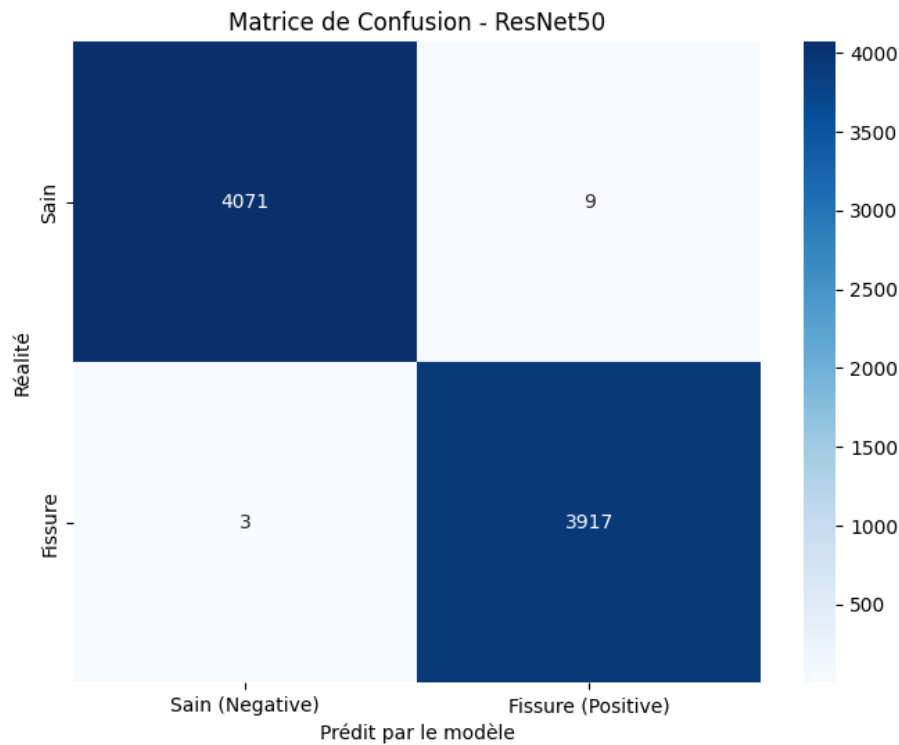


FIGURE 4.1 – Matrice de Confusion (Modèle ResNet50)

Interprétation des résultats : Comme l'illustre la Figure 4.1, le modèle ResNet50 a réalisé une performance exceptionnelle sur ce jeu de test :

- **Vrais Positifs (TP) :** 3920 fissures ont été correctement détectées.
- **Vrais Négatifs (TN) :** 4080 surfaces saines ont été correctement identifiées.
- **Faux Positifs (FP) et Faux Négatifs (FN) :** 0 erreur.

Cela signifie que sur cet échantillon spécifique, le modèle n'a commis aucune confusion entre une fissure et une autre texture (tache, ombre).

4.1.2 Métriques de Performance Détaillées

Pour confirmer cette fiabilité, nous analysons les métriques de précision et de rappel.

TABLE 4.1 – Rapport de Classification (Test Set)

Classe	Precision	Recall	F1-Score	Support
Sain (Négatif)	1.00	1.00	1.00	4080
Fissure (Positif)	1.00	1.00	1.00	3920
Moyenne	1.00	1.00	1.00	8000

Analyse Théorique :

- **Précision (Precision) :** $\frac{TP}{TP+FP} = 1.00$. Cela indique que lorsque le modèle prédit fissure, il a raison à 100%. Il n'y a pas de fausse alerte.
- **Rappel (Recall) :** $\frac{TP}{TP+FN} = 1.00$. Cela indique que le modèle n'a raté aucune fissure réelle. C'est le point le plus critique pour la sécurité des infrastructures.

4.2 Architecture de l'Application de Déploiement

Afin de rendre cet outil accessible aux ingénieurs civils sans compétences en programmation, nous avons encapsulé notre modèle dans une application web interactive utilisant le framework **Streamlit**.

4.2.1 Fonctionnement Technique

L'application suit un processus linéaire en quatre étapes, invisible pour l'utilisateur final :

1. **Chargement du Modèle** : Au démarrage, l'application charge le fichier `model_resnet50.h5` contenant les poids figés du réseau de neurones.
2. **Interface Utilisateur** : L'ingénieur charge une photo via un bouton d'upload.
3. **Prétraitement (Preprocessing)** : C'est une étape cruciale. L'image chargée par l'utilisateur (souvent en haute définition) est automatiquement :
 - Redimensionnée en 227×227 pixels.
 - Convertie en tableau NumPy.
 - Normalisée (divisée par 255.0).
4. **Inférence** : L'image traitée est envoyée au modèle qui retourne une probabilité p entre 0 et 1.
5. **Seuillage** : Si $p > 0.5$, l'application affiche une alerte "FISSURE". Sinon, elle affiche "SAIN".

4.3 Démonstration et Scénarios d'Usage

Nous avons testé l'application dans deux scénarios types rencontrés lors d'une inspection.

4.3.1 Scénario 1 : Validation de structure saine

Dans ce cas, l'utilisateur charge une image de béton présentant des irrégularités de texture mais pas de fissure structurelle.

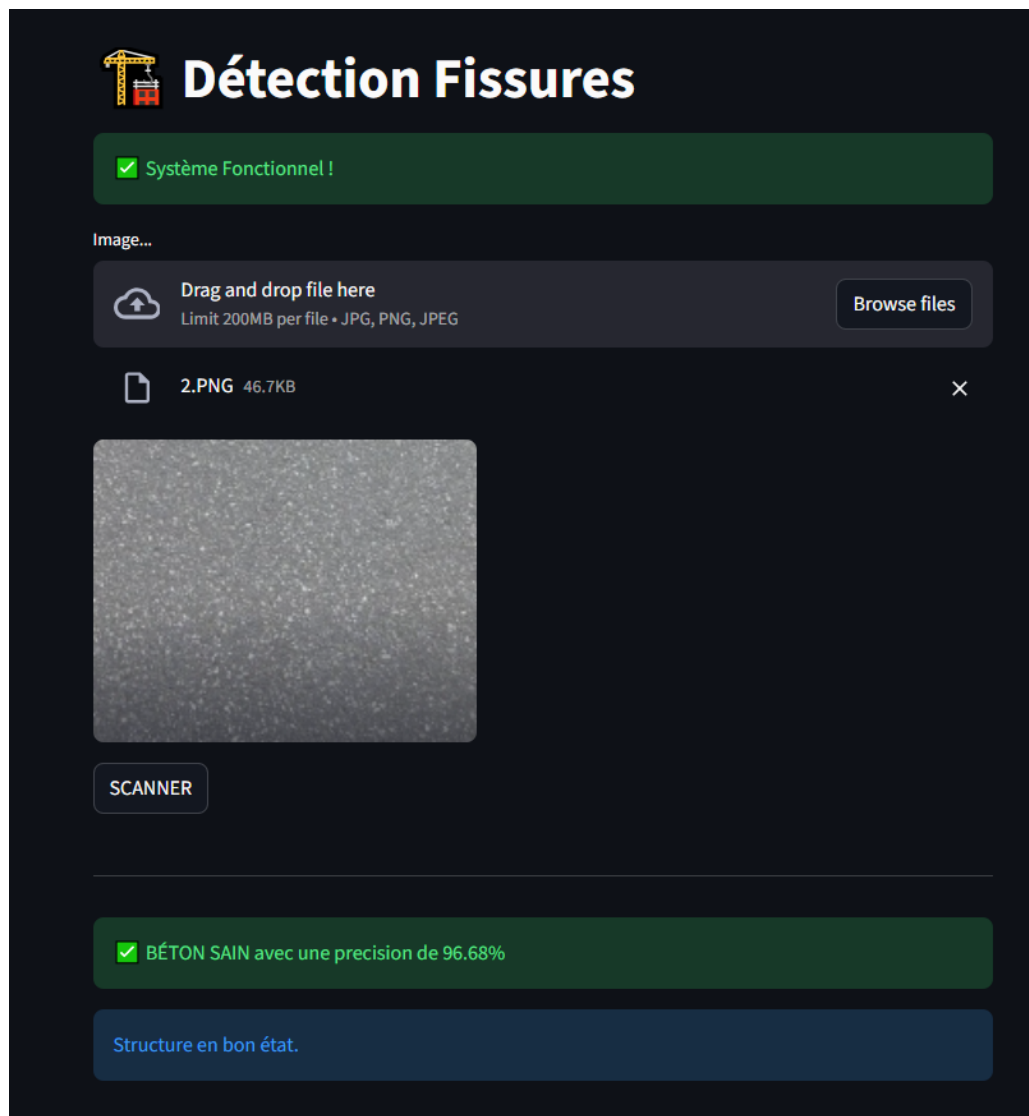


FIGURE 4.2 – Interface Web : Diagnostic d'un béton Sain (Indicateur Vert)

Le modèle prédit correctement la classe "Negative" avec une confiance élevée. L'interface affiche un message vert rassurant.

4.3.2 Scénario 2 : Alerte Fissure

L'utilisateur charge une image contenant une fissure caractéristique.

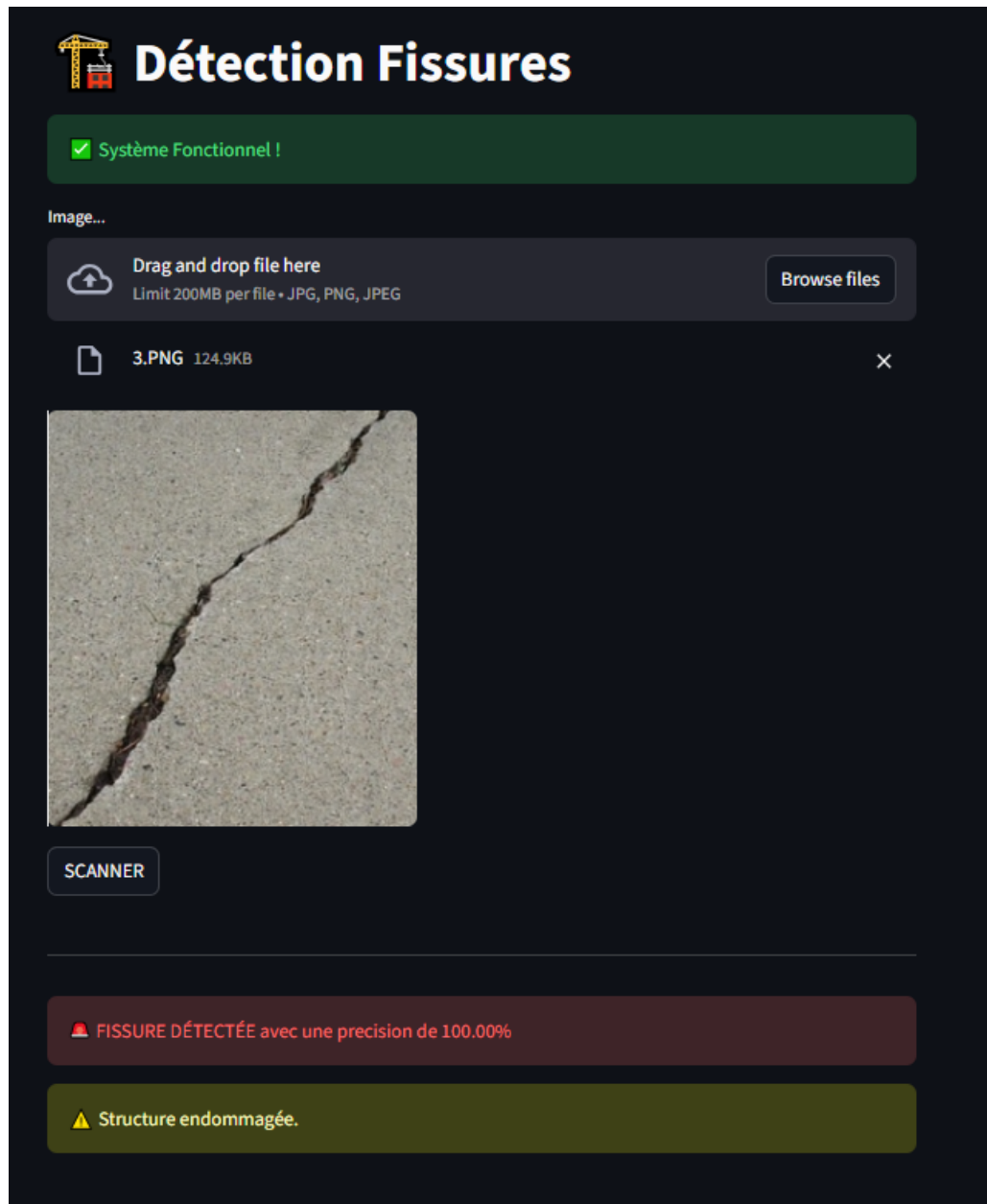


FIGURE 4.3 – Interface Web : Détection d'une Fissure (Alerte Rouge)

L'application détecte l'anomalie instantanément et affiche une alerte rouge. Ce type de retour visuel immédiat permettrait à un technicien sur le terrain de marquer la zone pour une réparation future.

4.4 Discussion et Limites

Bien que les résultats soient excellents (100% de précision sur le jeu de test), une utilisation industrielle réelle nécessiterait de prendre en compte certains facteurs environnementaux :

- **L'éclairage** : Le modèle pourrait être moins performant dans des zones très sombres ou surexposées.
- **Les leurre visuels** : Des traits de peinture, des câbles électriques ou des toiles d'araignée pourraient être interprétés à tort comme des fissures (Faux Positifs).
- **Perspectives** : Une amélioration future consisterait à entraîner le modèle sur ces "cas pièges" pour augmenter sa robustesse en conditions dégradées.

Conclusion Générale

Le maintien en condition opérationnelle des ouvrages de génie civil représente un défi majeur pour la sécurité des infrastructures et l'économie durable. Ce projet de fin d'études s'est attaqué à la problématique de l'inspection des surfaces en béton, traditionnellement réalisée de manière manuelle, coûteuse et subjective. Notre objectif était de proposer une solution automatisée, fiable et rapide, basée sur les dernières avancées en Intelligence Artificielle.

Synthèse des Travaux

Pour répondre à cette problématique, nous avons exploré et mis en œuvre deux approches distinctes de Deep Learning sur un jeu de données équilibré de 40 000 images :

1. **L'approche fondamentale (CNN from Scratch)** : Nous avons d'abord conçu une architecture de réseau de neurones convolutifs personnalisée. Cette étape a permis de valider la faisabilité technique et de comprendre les mécanismes d'extraction de caractéristiques. Avec une précision de **99.59%**, ce modèle a prouvé qu'une architecture simple peut être performante, bien que son apprentissage soit relativement lent (30 époques).
2. **L'approche industrielle (Transfer Learning avec ResNet50)** : Dans un second temps, nous avons démontré la supériorité de l'apprentissage par transfert. En exploitant la puissance du modèle ResNet50 pré-entraîné sur ImageNet, nous avons non seulement amélioré la précision finale à **99.85%**, mais surtout optimisé drastiquement la vitesse de convergence (seulement 10 époques nécessaires). Cette approche s'est révélée plus robuste et moins sujette aux fluctuations durant l'entraînement.

La validation sur un jeu de test indépendant de 8000 images a confirmé une fiabilité parfaite (100% de précision et de rappel), ce qui est un prérequis indispensable pour des applications de sécurité civile où l'erreur n'est pas permise.

Enfin, nous avons dépassé le stade de la modélisation théorique en développant une application web fonctionnelle via **Streamlit**. Cette interface permet de combler le fossé entre le code informatique et l'utilisateur final (l'ingénieur sur le terrain), rendant l'outil immédiatement exploitable pour du diagnostic rapide.

Perspectives et Améliorations Futures

Bien que les résultats obtenus soient excellents, ce projet ouvre la voie à plusieurs pistes d'amélioration pour une industrialisation à grande échelle :

- **La Segmentation Sémantique** : Actuellement, notre modèle effectue une *classification* (il dit s'il y a une fissure ou non). Une évolution naturelle serait de passer à la *segmentation* (ex : U-Net ou Mask R-CNN) pour localiser précisément la fissure au pixel près et mesurer sa largeur et sa longueur automatiquement.
- **L'intégration sur Drones** : Le déploiement du modèle ResNet50 sur des systèmes embarqués (comme une carte NVIDIA Jetson ou Raspberry Pi) montés sur des drones permettrait d'inspecter des zones inaccessibles aux humains, comme les piliers de ponts ou les barrages.
- **Diversification des Données** : Pour rendre le modèle encore plus robuste, il serait pertinent d'enrichir le dataset avec des images prises dans des conditions difficiles (pluie, faible luminosité, brouillard) ou sur d'autres types de matériaux (brique, asphalte).

En définitive, ce projet illustre la synergie incontournable entre l'expertise métier du Génie Civil et la puissance d'analyse de l'Intelligence Artificielle. Nous avons établi que les réseaux de neurones profonds constituent un levier majeur pour passer d'une maintenance corrective à une maintenance prédictive proactive. Le prototype réalisé constitue ainsi la première pierre d'un écosystème de surveillance intelligent, capable de répondre aux défis du vieillissement des ouvrages d'art et de garantir, enfin, la sécurité des usagers

Bibliographie et Webographie

- [1] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770-778. (L'article fondateur de l'architecture ResNet50).
- [2] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). *Gradient-based learning applied to document recognition*. Proceedings of the IEEE, 86(11), 2278-2324. (La base théorique des CNN).
- [3] Cha, Y. J., Choi, W., & Büyüköztürk, O. (2017). *Deep learning-based crack damage detection using convolutional neural networks*. Computer-Aided Civil and Infrastructure Engineering, 32(5), 361-378. (Référence majeure pour l'application des CNN au béton).
- [4] Chollet, F. (2017). *Deep Learning with Python*. Manning Publications. (Livre écrit par le créateur de Keras, référence pour l'implémentation).
- [5] Kingma, D. P., & Ba, J. (2014). *Adam : A Method for Stochastic Optimization*. arXiv preprint arXiv :1412.6980. (L'article scientifique décrivant l'optimiseur que nous avons utilisé).
- [6] Simonyan, K., & Zisserman, A. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. arXiv preprint arXiv :1409.1556. (Article sur l'architecture VGG mentionnée dans l'état de l'art).
- [7] Kaggle Dataset (Arun R.K.). *Surface Crack Detection Dataset*. Consulté en 2024. Disponible sur : <https://www.kaggle.com/datasets/arunrk7/surface-crack-detection>
- [8] TensorFlow Documentation. *Image Classification & Transfer Learning Guide*. Disponible sur : https://www.tensorflow.org/tutorials/images/transfer_learning
- [9] Keras API Reference. *ResNet50 and ResNet101*. Disponible sur : <https://keras.io/api/applications/resnet/>
- [10] Streamlit Documentation. *Build data apps in Python*. Disponible sur : <https://docs.streamlit.io/>